

HashiCorp Vault Pentest — Exact Command Reference

Companion to `vault_pentest_checklist.md`. Section numbers match. Assumes `VAULT_ADDR` is exported. Where a command needs a specific identity, it's labeled `[READER]` or `[ADMIN]`. Replace `<...>` placeholders.



```
bash
export VAULT_ADDR="https://vault.example.com:8200"
```

0. Pre-Engagement / Recon



```
bash
# Version / build info (often exposed pre-auth)
curl -sk $VAULT_ADDR/v1/sys/health | jq

vault status

# Full health with standby/perf info
curl -sk "$VAULT_ADDR/v1/sys/health?standbyok=true&perfstandbyok=true" | jq

# HA / leader info
curl -sk $VAULT_ADDR/v1/sys/leader | jq

# Seal status (unauthenticated)
curl -sk $VAULT_ADDR/v1/sys/seal-status | jq
vault status -format=json | jq '.sealed, .type, .cluster_name'
```

Cross-reference the `version` field against CVE databases (`searchsploit vault`, NVD, GitHub advisories for `hashicorp/vault`).

1. TLS & Network Layer



```
bash
```

```
# TLS cert/cipher inspection
nmap --script ssl-enum-ciphers -p 8200 vault.example.com
openssl s_client -connect vault.example.com:8200 -tls1_2 </dev/null
openssl s_client -connect vault.example.com:8200 -tls1 </dev/null # should fail

# Cluster port exposure check (should NOT be reachable externally)
nmap -p 8201 vault.example.com

# CORS config [ADMIN]
vault read sys/config/cors
curl -sk -H "X-Vault-Token: $ADMIN_TOKEN" $VAULT_ADDR/v1/sys/config/cors | jq

# Test CORS from attacker-controlled origin
curl -sk -H "Origin: https://evil.example.com" -I $VAULT_ADDR/v1/sys/health

# Rate limit quotas [ADMIN]
vault read sys/quotas/rate-limit/<name>
vault list sys/quotas/rate-limit

# Manual brute-force / rate-limit test (authorized load only, coordinate with client)
for i in $(seq 1 50); do
  curl -sk -X POST $VAULT_ADDR/v1/auth/userpass/login/testuser \
    -d '{"password":"wrong"}' -o /dev/null -w "%{http_code}\n"
done
```

2. Unauthenticated Enumeration

```


bash
curl -sk $VAULT_ADDR/v1/sys/health
curl -sk $VAULT_ADDR/v1/sys/seal-status
curl -sk $VAULT_ADDR/v1/sys/leader

# These SHOULD all return 403/permission denied unauthenticated — confirm
curl -sk $VAULT_ADDR/v1/sys/mounts
curl -sk $VAULT_ADDR/v1/sys/policies/acl
curl -sk $VAULT_ADDR/v1/sys/auth
curl -sk $VAULT_ADDR/v1/secret/data/anything
```

3. Authentication Methods Audit



bash

List all auth methods [READER or ADMIN — reader may get 403, note it]

```
vault auth list -detailed
```

```
curl -sk -H "X-Vault-Token: $TOKEN" $VAULT_ADDR/v1/sys/auth | jq
```

--- OIDC / JWT ---

List OIDC/JWT roles [ADMIN]

```
vault list auth/oidc/role
```

```
vault read auth/oidc/role/<role-name>
```

```
curl -sk -H "X-Vault-Token: $ADMIN_TOKEN" $VAULT_ADDR/v1/auth/oidc/role/<role-name> | jq
```

Inspect for: bound_audiences, bound_claims, bound_claims_type,

allowed_redirect_uris, oidc_scopes, token_policies, token_ttl, token_max_ttl

```
vault read -format=json auth/oidc/role/<role-name> | jq '.data | {bound_audiences, bound_claims, bound_claims_type, allowed_redirect_uris, oidc_scopes, token_policies, token_ttl, token_max_ttl}'
```

Global OIDC config [ADMIN]

```
vault read auth/oidc/config
```

Perform actual OIDC login as reader (via CLI, opens browser flow)

```
vault login -method=oidc role="<role-name>"
```

Inspect the token you get back — decode policies/claims

```
vault token lookup -format=json | jq '.data | {policies, meta, entity_id, identity_policies}'
```

If you have the raw ID token / access token, decode the JWT claims locally

```
echo "<jwt>" | cut -d ' ' -f2 | base64 -d 2>/dev/null | jq
```

Test redirect_uri manipulation (open redirect / token theft) — manually craft

the OIDC auth URL with a modified redirect_uri and see if Vault's callback

still accepts it:

```
vault write auth/oidc/oidc/auth_url role="<role-name>" redirect_uri="http://attacker.example.com/callback"
```

Test claims_type glob abuse — as admin, check the raw glob pattern:

```
vault read -field=bound_claims auth/oidc/role/<role-name>
```

Then attempt login with an IdP identity that technically matches the glob

but shouldn't be intended (coordinate with IdP test account provisioning)

--- AppRole ---

```
vault list auth/approle/role
```

```
vault read auth/approle/role/<role-name>
```

```
vault read auth/approle/role/<role-name>/role-id
```

Check secret-id constraints

```

vault read -format=json auth/approle/role/<role-name> | jq '.data | {secret_id_bound_cidrs, secret_id_num_us

# Try to read/generate secret-id as reader if policy allows (shouldn't)
vault write -f auth/approle/role/<role-name>/secret-id

# --- Kubernetes auth ---
vault list auth/kubernetes/role
vault read -format=json auth/kubernetes/role/<role> | jq '.data | {bound_service_account_names, bound_serv

# --- Userpass / LDAP ---
vault read auth/userpass/users/<user>
vault read auth/ldap/config
vault read sys/mfa/method # MFA enforcement check
vault read sys/mfa/login-enforcement/<name>
```

4. Policy (ACL) Review



bash

List and dump every policy [ADMIN; reader will 403 on most of these — try anyway and note]

```
vault policy list
for p in $(vault policy list); do
  echo "=== $p ==="
  vault policy read "$p"
done
```

Save all policies to files for offline grepping

```
mkdir -p policies && for p in $(vault policy list); do
  vault policy read "$p" > "policies/$p.hcl"
done
```

Grep for dangerous patterns across all policies

```
grep -rn 'capabilities.*sudo' policies/
grep -rln ""\*" policies/
grep -rn 'sys/policies/acl' policies/
grep -rn 'sys/auth' policies/
grep -rn 'sys/mounts' policies/
grep -rn 'auth/token/create' policies/
grep -rn 'identity/' policies/
grep -rn 'sys/raw' policies/
grep -rn 'sys/leases' policies/
```

--- THE most important test: actual effective capabilities as READER ---

Run this as the reader token against every sensitive path you find:

```
vault token capabilities <reader-token> sys/policies/acl/admin
vault token capabilities <reader-token> auth/token/create
vault token capabilities <reader-token> identity/entity
vault token capabilities <reader-token> identity/entity-alias
vault token capabilities <reader-token> identity/group
vault token capabilities <reader-token> sys/mounts
vault token capabilities <reader-token> sys/raw/
vault token capabilities <reader-token> sys/leases/revoke
```

Or self-check without needing to pass the token explicitly (uses VAULT_TOKEN env/current login)

```
vault token capabilities sys/policies/acl/admin
vault token capabilities auth/token/create
vault token capabilities identity/entity
```

API equivalent (batch check many paths at once)

```
curl -sk -H "X-Vault-Token: $READER_TOKEN" $VAULT_ADDR/v1/sys/capabilities-self \
```

```
-d '{"paths": ["sys/policies/acl/admin", "auth/token/create", "identity/entity", "sys/mounts", "sys/raw/"]}' | jq
```

Identity groups the reader belongs to

```
vault read identity/entity/id/$(vault token lookup -field=entity_id)
```

5. Token & Lease Management



bash

Check root token usage / existence patterns [ADMIN]

```
vault token lookup          # check your own
```

```
vault list auth/token/accessors  # enumerate all live token accessors [ADMIN]
```

```
for acc in $(vault list -format=json auth/token/accessors | jq -r '[]'); do
```

```
echo "=== $acc ==="
```

```
vault token lookup -accessor "$acc"
```

```
done
```

Test orphan token creation as reader

```
vault token create -orphan -policy="<reader-policy>"
```

Test token renewal beyond expected max (as reader, on own token)

```
vault token renew
```

```
vault token renew -increment=87600h <token>  # try absurd increment, see if capped correctly
```

Test lease enumeration/revocation authZ as reader (should fail unless authorized)

```
vault list sys/leases/lookup/
```

```
curl -sk -H "X-Vault-Token: $READER_TOKEN" $VAULT_ADDR/v1/sys/leases/lookup/secret/data/somepath
```

Try revoking a lease that isn't yours

```
vault lease revoke <lease-id-belonging-to-other-identity>
```

Check periodic token behavior

```
vault token create -policy="<policy>" -period=768h
```

```
vault token renew <periodic-token>  # confirm it renews indefinitely — flag if unrestricted
```

token_num_uses test

```
vault token create -policy="<policy>" -use-limit=1
```

```
vault token lookup <that-token>  # use twice, confirm second use fails
```

6. Secrets Engines Review



bash


```
vault read transit/export/encryption-key/<key-name>

# Test encrypt/decrypt authZ as reader on a key they shouldn't touch
echo -n "test" | base64 | xargs -I{} vault write transit/encrypt/<key-name> plaintext={}

# --- Cloud secrets engines (AWS example) ---
vault read aws/config/root      # check if root/IAM creds retrievable (shouldn't be readable, only writable)
vault list aws/roles
vault read aws/roles/<role>
vault read aws/creds/<role>     # check STS scoping / TTL

# --- SSH secrets engine ---
vault list ssh/roles
vault read ssh/roles/<role>
```

7. Identity, Entities & Groups



bash
vault list identity/entity/id
vault list identity/entity-alias/id
vault list identity/group/id
vault list identity/group/name

vault read identity/group/name/<group-name>
vault read -format=json identity/group/name/<group-name> | jq '.data | {policies, member_entity_ids, member_entity_aliases}'

Read your own entity as reader
vault read identity/entity/id/\${vault token lookup -field=entity_id}

Test self-modification of entity alias (should fail for reader)
vault write identity/entity-alias id=<alias-id> canonical_id=<your-entity-id> mount_accessor=<accessor> name=<name>

Test entity merge abuse [needs identity/entity/merge capability — check first]
vault token capabilities identity/entity/merge
vault write identity/entity/merge from_entity_ids=<low-priv-id> to_entity_id=<high-priv-id>

8. Audit Logging



bash

```
vault audit list -detailed
```

```
curl -sk -H "X-Vault-Token: $ADMIN_TOKEN" $VAULT_ADDR/v1/sys/audit | jq
```

Enable a test audit device temporarily if none exist and it's in scope to check behavior

```
vault audit enable file file_path=/tmp/vault_audit_test.log
```

Confirm HMAC-only on sensitive fields — inspect a sample log entry

```
tail -n 5 /tmp/vault_audit_test.log | jq '.request.data, .response.data'
```

Values should show "hmac-sha256:..." not raw secrets

Test what happens when audit logging fails (ONLY in a lab/non-prod clone — this blocks all Vault requests by

9. Seal / Unseal & Recovery



bash

```
vault status -format=json | jq '.type, .sealed, .t, .n' # seal type, threshold, shares (Shamir)
```

Rekey process visibility [ADMIN]

```
vault operator rekey -status
```

Root token generation process check

```
vault operator generate-root -status
```

If auto-unseal (e.g. AWS KMS) — check the KMS key policy directly via cloud CLI

```
aws kms get-key-policy --key-id <key-id> --policy-name default
```

```
aws kms list-grants --key-id <key-id>
```

10. Storage Backend Direct Access



bash

Raft storage

`vault operator raft list-peers`

`vault read sys/storage/raft/configuration`

Snapshot access control test

`vault operator raft snapshot save /tmp/vault.snap`

Check who else can reach this file / the API path

`vault token capabilities sys/storage/raft/snapshot`

Consul backend (if applicable) — test independently

`consul acl token list`

`consul kv get -recurse vault/`

11. Privilege Escalation Path Testing (chained, as READER)



bash

Step 1: full effective capability map

```
curl -sk -H "X-Vault-Token: $READER_TOKEN" $VAULT_ADDR/v1/sys/capabilities-self \
-d '{"paths": [
  "auth/token/create",
  "sys/policies/acl/admin",
  "sys/policies/acl/""$(vault token lookup -field=display_name)""",
  "identity/entity-alias",
  "identity/group",
  "pki/roles/*",
  "database/roles/*",
  "transit/keys/*"
]}' | jq
```

Step 2: attempt token creation with broader policy

```
vault login -no-print token=$READER_TOKEN
vault token create -policy="admin" -policy="root"
```

Step 3: attempt self policy edit (if update capability found in step 1)

```
echo 'path "*" { capabilities = ["read","list"] }' > /tmp/escalate.hcl
vault policy write <own-policy-name> /tmp/escalate.hcl
```

Step 4: attempt role tuning abuse on a secrets engine

```
vault write pki/roles/<reader-owned-role> allow_any_name=true max_ttl=87600h
```

Step 5: response-wrapping interception test (if wrap tokens are passed via

shared channel like Slack/email as part of onboarding — social/process test)

```
vault unwrap <intercepted-wrapping-token>
```

Document each successful step with `vault token lookup` output showing

policy/identity BEFORE and AFTER each escalation step.

12. Namespaces (Enterprise)



bash

```
vault namespace list
vault namespace lookup -namespace=<child-ns>
```

Cross-namespace access test

```
VAULT_NAMESPACE=<child-ns> vault token capabilities <root-ns-token-if-any> secret/data/child-secret
```

Replication status

```
vault read -format=json sys/replication/status | jq
```

13. Vault Agent / Proxy (host-level, if in scope)



bash

On a host running Vault Agent

```
find / -iname "*.hcl" 2>/dev/null | xargs grep -l "auto_auth" 2>/dev/null
```

```
cat /etc/vault-agent/agent.hcl # check auto_auth method, sink permissions
```

```
ls -la /etc/vault-agent/secret-id # check file perms — should be 0600, owned by service user
```

```
ls -la /var/run/secrets/vault/ # rendered template file perms
```

```
ps aux | grep vault-agent
```

```
lsof -p <agent-pid> # check open file handles for cached tokens
```

14. API / Misc Hardening



bash

```
vault read sys/config/ui
vault read sys/config/control-group

# sys/raw exposure test — this should ALWAYS fail for non-root
vault read sys/raw/sys/policy

# HTTP header info disclosure
curl -skI $VAULT_ADDR/v1/sys/health | grep -i vault

# pprof exposure
curl -sk $VAULT_ADDR/v1/sys/pprof/goroutine -o /tmp/pprof_test

# Plugin catalog / integrity check [ADMIN]
vault plugin list
vault read sys/plugins/catalog/secret/<plugin-name>
```

Handy: Save Everything for the Report



bash

```
mkdir -p vault_pentest_evidence
vault status -format=json > vault_pentest_evidence/status.json
vault policy list > vault_pentest_evidence/policy_list.txt
vault auth list -format=json > vault_pentest_evidence/auth_methods.json
vault secrets list -format=json > vault_pentest_evidence/secrets_engines.json
vault audit list -format=json > vault_pentest_evidence/audit_devices.json
```